Programming Assignment 3 | Team 14

Andrew Goodson, Elijah Johnson, Sophie Shah, Nicholas Lindner

Due Wednesday, April 22, 12:50 PM on Canvas

Youtube Demo

https://youtu.be/gph4gPyhe3s

Software Design

This project extends the functionality from PA1 (basic client-server communication) and PA2 (file transfers) by introducing concurrent clients and batched file transmissions. Experiments were conducted using multiple concurrent clients (1, 2, 4, and 8) to observe the impact of contention on server performance.

Architecture

The system follows a multi-thread client-server model. The server runs continuously and listens for incoming connections. For each client connection, the server then creates a dedicated handler thread that allows it to support multiple clients concurrently. Each client independently connects, requests files, and measures its RTT.

Server-Client Behavior

After connecting and receiving a "Hello!", the client starts a timer, sends "SEND" to the server, generates a random permutation of integers, sends the ordering to the server, receives 10 images in batches, saves images to a created directory, and finally stops the timer and computes RTT. This process is repeated at least 5 times to automatically calculate statistics.
The server sends a "Hello!" after client connection. Once it receives a "SEND", it responds with "sendnums", receives the randomized image order and loads them, and sends the images in batches. The batch sizes dynamically cycle through. Each client is handled by a separate thread, which ensures the concurrency needed in this project.

Batching Implementation

The ImageFileBatch class is used to encapsulate multiple images. Each batch contains a list of ImageFile objects, storing the filename, size, and raw byte data. This allows for the transfer of multiple files per network operation and reduces communication overhead compared to the previous PA2.

Round-Trip Time Statistics at the Client

RTT measurements were collected across batch sizes 1, 2, and 3, with batch size dynamically changing per request.

Statistic	Time (ms)
Mean	56876.8

Statistic	Time (ms)
Median	58449
Mode	N/A
Minimum	50881
Maximum	63014
Standard Deviation	4465.38

Comparison with PA2 RTT Statistics

Statistic	Time (ms)
Mean	1003
Median	1018
Mode	N/A
Minimum	491
Maximum	1350
Standard Deviation	318.07

Key Differences:

1. Significant increase in RTT in PA3. This is expected because PA3 involves transferring 10 images in batches whereas PA2 involved transferring files individually with less workload per request.
2. Higher performance variability, shown by the increase in standard deviation. This shows that the RTT is much more variable in PA3, likely due to batching differences, I/O overhead, and added complexity of handling multiple images in a single request.
3. Significantly heavier workload in PA3, shown by how even the fastest case in PA3 is slower than the slowest case in PA2.
4. Visualized batching overhead, as seen by the significantly increased RTT compared to PA2. While batching reduces the number of network transmissions, it adds overhead from larger memory usage, writing many files to the disk at once, and the serialization of multiple files.

Lessons Learned

We learned a lot about concurrency, batching, and network performance in this programming assignment. PA3 showed how handling multiple clients simultaneously introduces contention on the server which can increase RTT even when using more efficient communication methods. We learned how batching files together can improve the overall performance by reducing communication overhead since fewer messages are sent between the client and server. At the same time, we saw how increasing the batch size can introduce different tradeoffs for latency and throughput. We also learned how multithreading allows for servicing multiple clients concurrently, and how proper thread management is necessary to avoid resource issues.